

栈、队列

栈的定义和操作

- 栈只允许在一段进行插入或删除操作的线性表
 - 先进后出, 后进先出
- 特点
 - 栈顶: 允许插入删除的一端
 - 栈底: 不允许插入删除的一端
 - 空栈: 不含任何元素的空表
- 考点: 入栈序列和出栈序列之间的关系
 - ① InitStack(&s): 初始化一个空栈
 - ② StackEmpty(S): 判断栈是否为空, 返回布尔值
 - ③ Push(&S,x): 进栈, 把元素x压入栈顶
 - ④ Pop(&S,&x): 出栈, 元素x弹出栈顶
 - ⑤ GetTop(S,&x): 读取栈顶元素, 用x返回栈顶元素
- 栈的数学性质
 - n个不同元素进栈, 出栈元素不同的排列个数为 $\frac{1}{n+1}C_{2n}^n$

重点记住, 做选择题使用

栈的顺序存储结构

- 顺序栈的实现

```
typedef struct {
    ElemType data[MaxSize]; //存放栈顶元素
    int top; //栈顶指针top
} SqStack;
```
- 顺序栈的基本操作
 - 初始化: S.top = -1; 栈顶元素: S.data[S.top]
 - △ 有部分教材会将栈顶指针S.top定义为0, 相当于规定top指向栈顶元素的下一个存储单元
 - 栈空条件: S.top == -1
 - 栈满条件: S.top == MaxSize-1
 - 栈长: S.top + 1
 - 进栈操作
 - 栈不满时, 栈顶指针S.top先加1, 再送元素x到栈顶元素
 - S.data[++S.top] = x; //指针加1, 再进栈
 - 出栈操作
 - 栈非空时, 先取栈顶元素值x, 再将栈顶指针S.top先减1
 - x = S.data[S.top--]; //先把元素出栈, 指针再-1
 - 读取栈顶元素
 - x = S.data[S.top]
- 共享栈
 - 两个顺序栈共享存储空间, 两个栈顶向中间延伸
 - top0 = -1时0号栈为空, top1 = MaxSize时1号栈为空
 - 栈满条件: top1 - top0 = 1
 - 出栈是[-S.top]
 - 共享栈是为了更有效地利用存储空间, 两个栈空间相互调节, 只有在整个存储空间被占满时才发生上溢。
 - 存取数据的时间复杂度为O(1)

栈的链式存储结构

- 链表的基本结构
 - 头节点 栈顶节点 栈底节点
- 采用链式存储结构的栈称为链栈
 - 优点: 便于多个栈共享存储空间和提高效率, 不会出现栈满上溢的情况
 - 通常采用单链表实现, 便于结点的插入与删除基本操作和链表类似, 入栈和出栈的操作都在表头进行

队列的定义和基本操作

- 定义
 - 队列 (Queue) 只允许在表的一端进行插入, 而在表的另一端进行删除
 - 一种操作受限的线性表
 - 操作特性: 先进先出(FIFO)
- 基本操作
 - ① InitQueue(&Q): 初始化队列, 构造一个空队列
 - ② QueueEmpty(Q): 判队列空, 若队列为空返回 true, 否则返回 false
 - ③ EnQueue(&Q,&x): 入队, 若队列未满, 将x加入, 使之成为新队尾
 - ④ DeQueue(&Q,&x): 出队, 若队列非空, 删除对头元素, 返回x
 - ⑤ GetHead(Q &x) : 读队头元素 若队列 非空, 则将队头元素赋值给 X

队列的顺序存储结构

- 队列的顺序存储
 - 思想: 设置两个指针
 - front 指向对头元素
 - rear 指向队尾元素的下一个位置
 - 操作
 - 入队操作: 队不满时, 先送至队尾元素, 再将队尾指针 +1
 - 出队操作: 队不空时, 先取对头元素值, 再将队头指针 +1
 - 初始状态 (队空条件): Q.front == Q.rear == 0
 - 队长: rear - front
 - △ 假上溢: 不能用rear = MaxSize 判断队满
- 循环队列
 - 思想: 将顺序队列构造成一个环状的空间, 把存储元素的表在逻辑上看成环
 - 初始化: Q.front = Q.rear = 0
 - 队首指针进: Q.front = (Q.rear + 1) % MaxSize
 - 队尾指针进: Q.rear = (Q.rear + 1) % MaxSize
 - 队满条件: Q.rear = (Q.rear + 1) % MaxSize == front
 - 队空条件仍: Q.front == Q.rear
 - 队列元素个数: (Q.rear + MaxSize - Q.front) % MaxSize

队列的链式存储

- 链式存储
 - 缺点: 不带头结点的链式存储
 - front 指向对头元素
 - rear 指向对头元素的下一个位置
 - 入队: 队不满时, 先送至队尾元素, 再将队尾指针 +1
 - 出队: 队不空时, 先取对头元素值, 再将队头指针 +1
 - 初始状态 (队空条件): Q.front == Q.rear == 0
 - 队长: rear - front
 - △ 假上溢: 不能用rear = MaxSize 判断队满
 - 带头结点的链式存储
 - front 指向对头元素
 - rear 指向对头元素的下一个位置
 - 入队: 队不满时, 先送至队尾元素, 再将队尾指针 +1
 - 出队: 队不空时, 先取对头元素值, 再将队头指针 +1
 - 初始状态 (队空条件): Q.front == Q.rear == 0
 - 队长: rear - front
 - △ 假上溢: 不能用rear = MaxSize 判断队满
- 链式队列的基本操作
 - 入队
 - if (Q.rear + 1 % MaxSize == Q.front) return false; //判断是否队满
 - Q.data[Q.rear] = x; //向队尾插入x
 - Q.rear = (Q.rear + 1) % MaxSize; //队尾指针 +1 取模
 - 出队
 - if (Q.front == Q.front) return false; //判断是否队空
 - x = Q.data[Q.front]; //元素出队
 - Q.front = (Q.front + 1) % MaxSize; //对头指针加1取模
- 区分队空队满的三种操作
 - 牺牲一个单元区分队空和队满
 - 类型中增设size数据成员, 表示元素个数
 - 若删除成功, 则size-1
 - 若插入成功, 则size+1
 - 类型中增设tag数据成员, 以区分是队满还是队空

队列的链式存储

- 链式存储
 - 缺点: 不带头结点的链式存储
 - front 指向对头元素
 - rear 指向对头元素的下一个位置
 - 入队: 队不满时, 先送至队尾元素, 再将队尾指针 +1
 - 出队: 队不空时, 先取对头元素值, 再将队头指针 +1
 - 初始状态 (队空条件): Q.front == Q.rear == 0
 - 队长: rear - front
 - △ 假上溢: 不能用rear = MaxSize 判断队满
 - 带头结点的链式存储
 - front 指向对头元素
 - rear 指向对头元素的下一个位置
 - 入队: 队不满时, 先送至队尾元素, 再将队尾指针 +1
 - 出队: 队不空时, 先取对头元素值, 再将队头指针 +1
 - 初始状态 (队空条件): Q.front == Q.rear == 0
 - 队长: rear - front
 - △ 假上溢: 不能用rear = MaxSize 判断队满
- 链式队列的基本操作
 - 入队
 - 建立一个新结点, 将新结点插入链表的队尾, 并让rear指针指向新结点
 - s->data = x; s->next = null; //创建新结点, 插入到队尾
 - Q.rear->next = s; //将rear指针所指的结点的后继指针指向新结点
 - Q.rear = s; //rear指针后移到新结点
 - 出队
 - 首先判断是否为空, 若不为空则队头元素出队, 将其从链表中删除; 并让front指向下一个结点
 - 若队列中只有一个结点, 则令 Q.rear == null && Q.front == null;
 - if (Q.front == Q.rear) return false; //队空
 - LinkNode *p = Q.front->next; //p是指向被删除结点的指针
 - 被删除结点是头结点之后的结点
 - x = p->data; //被删除的元素值
 - Q.front->next = p->next; //头结点的后继指针指向被删除结点的后面的结点
 - if (Q.rear == p) Q.rear = Q.front; //若原队列只有一个元素, 删除后队列为空
 - free(p)
- 双端队列
 - 基础概念
 - 允许两端都可以进行入队和出队操作的队列
 - 输出受限
 - 允许一端插入删除, 另一端只允许插入
 - 输入受限
 - 允许一端插入删除, 另一端只允许删除

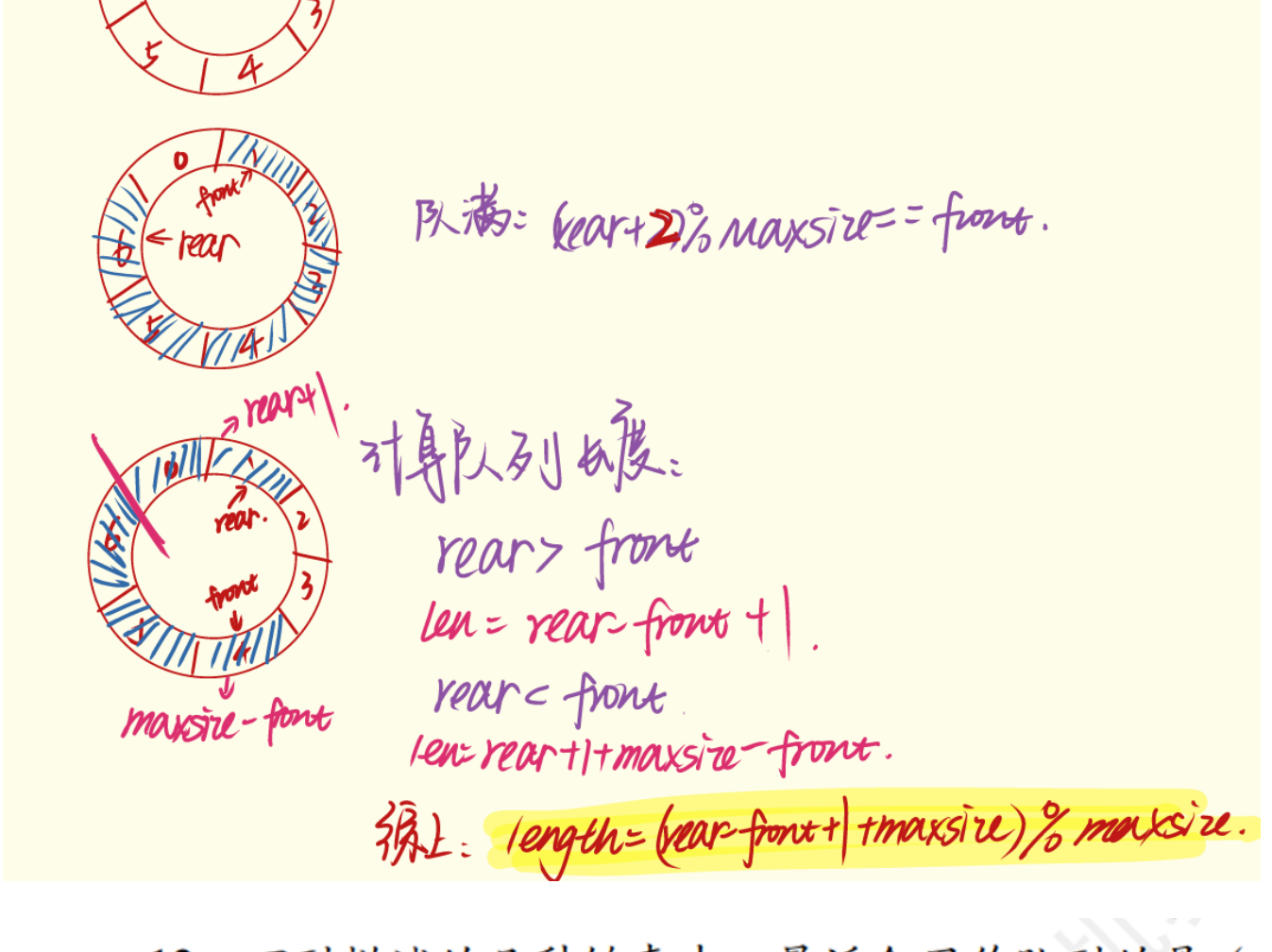
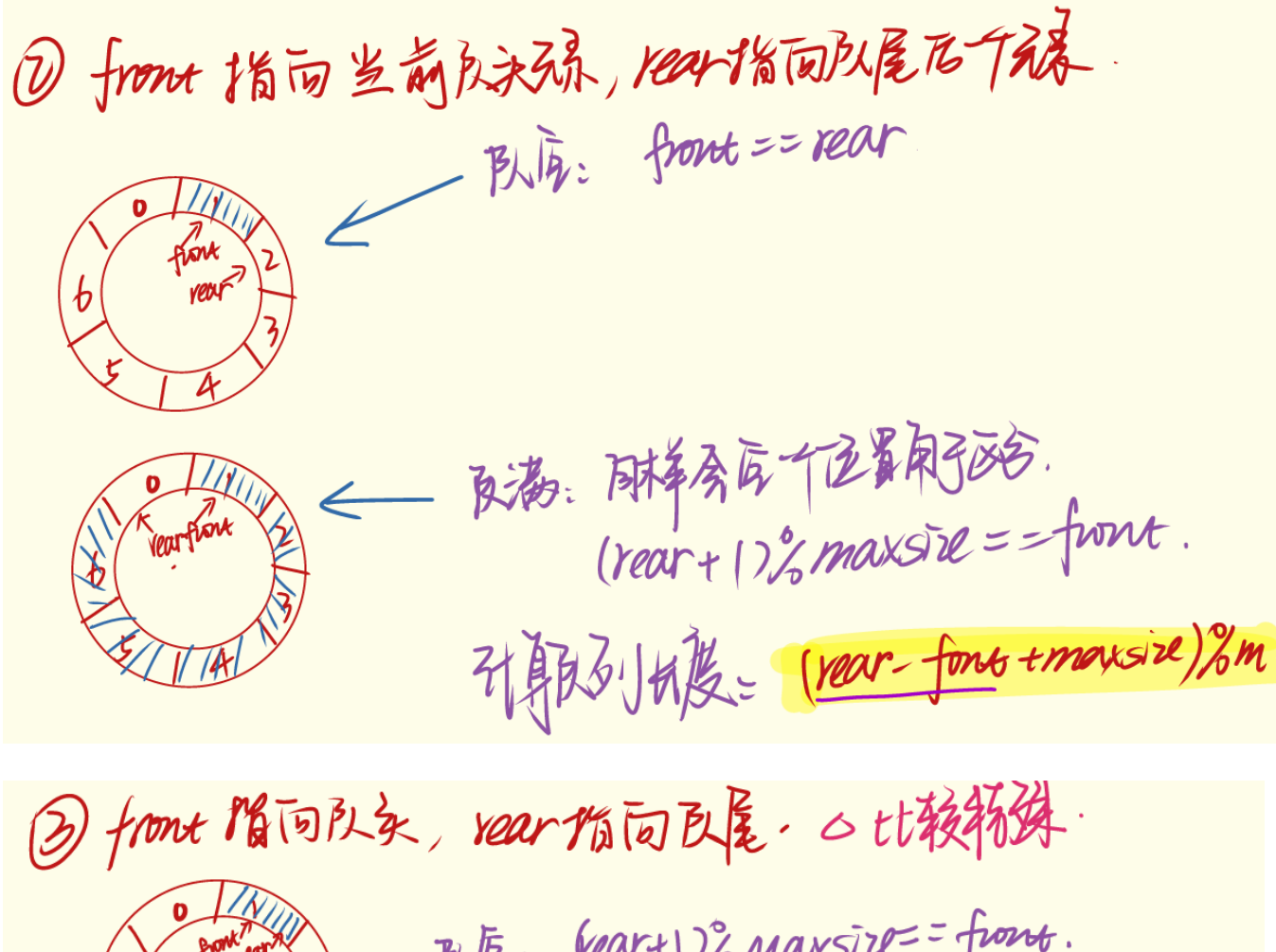
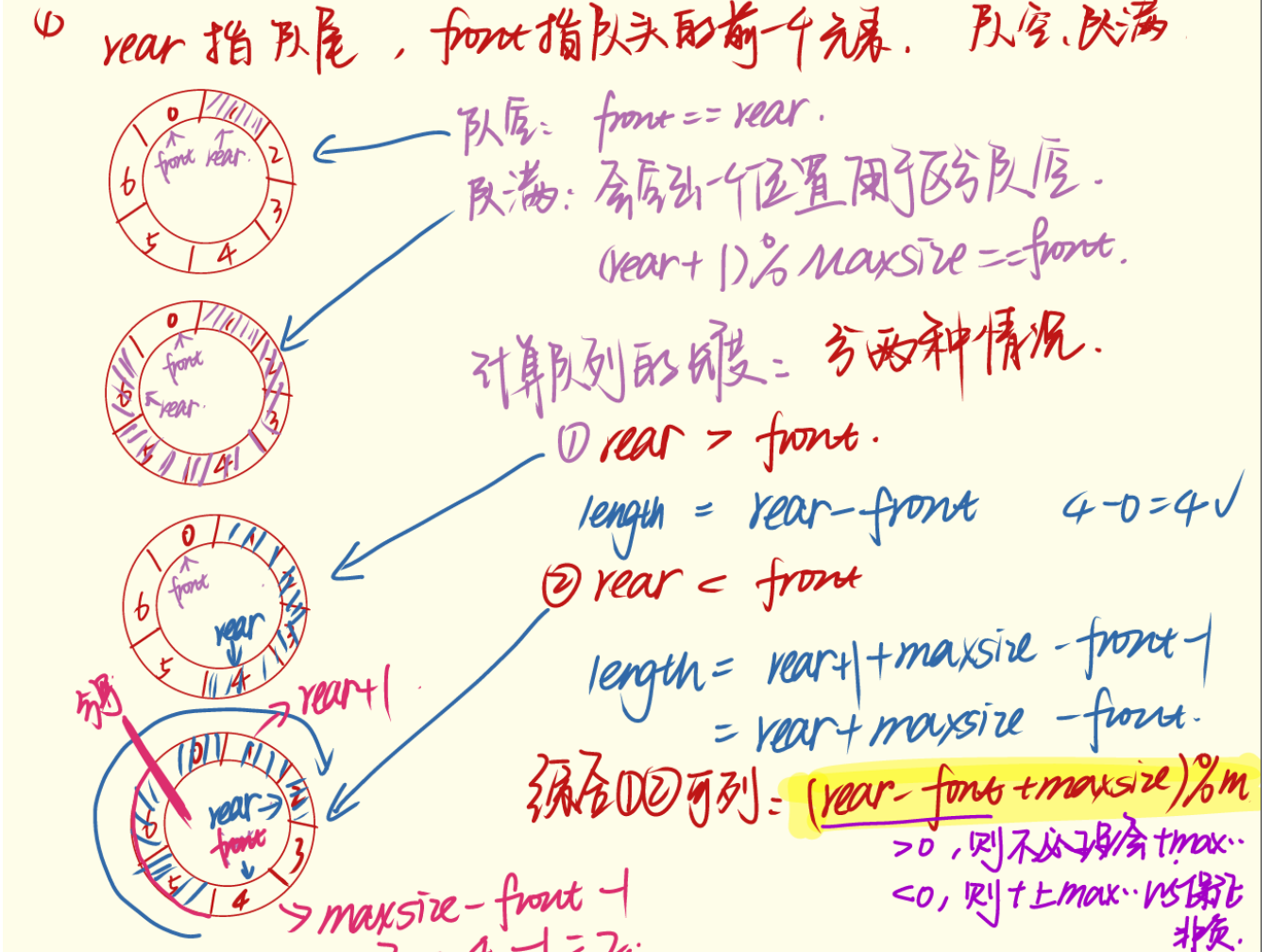
习题

栈

- 入栈出栈次序
 - 技巧: 画图模拟
 - 问栈的容量: 同样模拟操作, 看看栈中元素最多时是多少
- 链表实现的栈
 - 根据操作选择用什么链表实现链栈
 - 关于链栈的题目都把重心聚焦在链头的删除和链尾的插入
- 出栈序列的个数
 - 给出输出序列的某个元素, 判断+1的个数或者可能性这种题目
 - 第一种是取不到的比较少, 能取到的占大头, 则先计算所有可能取值 (用公式计算), 然后减去取不到的
 - 第二种就是可以取到的比较少, 只能找出每种可能的情况了
 - 有条件
 - 22. 已知栈的入栈序列是 1, 2, 3, 4, 其出栈序列为 P₁, P₂, P₃, P₄, 则 P₂, P₃ 不可能是 ()。
A. 2, 4 B. 2, 1 C. 4, 3 D. 3, 4
 - 29. 【2013 统考真题】一个栈的入栈序列为 1, 2, 3, ..., n, 出栈序列为 P₁, P₂, P₃, ..., P_n。若 P₂ = 3, 则 P₃ 可能取值的个数是 ()。
A. n-3 B. n-2 C. n-1 D. 无法确定
 - 无条件
 - n个不同元素进栈, 出栈元素不同的排列个数为 $\frac{1}{n+1}C_{2n}^n$

队列

- 判空判满的条件
 - 22. 【2014 统考真题】循环队列存放在一维数组 A[0..M-1] 中, end1 指向队首元素, end2 指向队尾元素的后一个位置, 假设队两端均可进行入队和出队操作, 队列中最多能容纳 M-1 个元素。初始时为空。下列判断队空和队满的条件中, 正确的是 ()。
A. 队空: end1==end2; 队满: end1==(end2+1) mod M
B. 队空: end1==end2; 队满: end2==(end2+1) mod (M-1)
C. 队空: end2==(end2+1) mod M; 队满: end1==(end2+1) mod M
D. 队空: end1==(end2+1) mod M; 队满: end2==(end2+1) mod (M-1)
- 队列的长度
 - 06. 已知循环队列的存储空间为数组 A[21], front 指向队首元素的前一个位置, rear 指向队尾元素, 假设当前 front 和 rear 的值分别为 8 和 3, 则该队列的长度为 ()。
A. 5 B. 6 C. 16 D. 17



- 链表的实现
 - 12. 下列描述的几种链表中, 最适合用作队列的是 ()。
A. 带队首指针和队尾指针的循环单链表
B. 带队首指针和队尾指针的非循环单链表
C. 只带队首指针的非循环单链表
D. 只带队首指针的循环单链表

- 双端队列
 - 像这种问题谁更合适, 也是像队一个, 聚焦于操作, 即看哪个数据结构插入删除更方便
 - 出队入队的次序问题
 - 这种题目就找第一个入队的序列, 看它在出队序列的哪里, 像汉堡一样夹着它

- 与栈相关的问题
 - 24. 【2021 统考真题】初始为空的队列 Q 的一端仅能进行入队操作, 另外一端既能进行入队操作又能进行出队操作。若 Q 的入队序列是 1, 2, 3, 4, 5, 则不能得到的出队序列是 ()。
A. 5, 4, 3, 1, 2 B. 5, 3, 1, 2, 4 C. 4, 2, 1, 3, 5 D. 4, 1, 3, 2, 5
 - 23. 【2018 统考真题】现有队列 Q 与栈 S, 初始时 Q 中的元素依次是 1, 2, 3, 4, 5, 6 (1 在队头), S 为空。若仅允许下列 3 种操作: ① 出队并输出出队元素; ② 出队并将出队元素入栈; ③ 出栈并输出出栈元素, 则不能得到的输出序列是 ()。
A. 1, 2, 5, 6, 4, 3 B. 2, 3, 4, 5, 6, 1 C. 3, 4, 5, 6, 1, 2 D. 6, 5, 4, 3, 2, 1